# 💻 Coding Problems, Python ML & Behavioral Q&A

---

## 📌 Section 1: Python & Coding for Written Exam

> Based on ReliSource's pattern (maze/path problems, algorithm problems), expect problem-solving questions. Practice these patterns.

---

### Pattern 1: BFS / DFS on Grids (Most likely ReliSource ML exam pattern)

**ReliSource gave a grid/maze problem before. For ML role, expect a similar logic problem in Python.**

**Problem: Shortest Path in a Grid**

Find the shortest path from top-left to bottom-right. `0` = blocked, `1` = open.

```
from collections import deque

def shortest_path(grid):
    if not grid or grid[0][0] == 0:
        return -1

    rows, cols = len(grid), len(grid[0])
    queue = deque([(0, 0, 1)])  # (row, col, distance)
    visited = set([(0, 0)])
    directions = [(0,1),(0,-1),(1,0),(-1,0)]

    while queue:
        r, c, dist = queue.popleft()

        if r == rows-1 and c == cols-1:
            return dist

        for dr, dc in directions:
            nr, nc = r+dr, c+dc
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1 ar
```

```
                visited.add((nr, nc))
                queue.append((nr, nc, dist+1))

    return -1

# Test
grid = [[1,1,1],[1,0,1],[1,1,1]]
print(shortest_path(grid))  # 4
```

---

**Problem: Longest Path in a Grid (ReliSource's exact style)**

```
def longest_path(grid):
    """
    0 = not traceable, 1 = traceable, other = destination
    Find longest path from top-left to destination
    """
    rows, cols = len(grid), len(grid[0])
    dest = None

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] not in (0, 1):
                dest = (r, c)

    if dest is None:
        return -1

    # DFS with backtracking
    visited = [[False]*cols for _ in range(rows)]
    directions = [(0,1),(0,-1),(1,0),(-1,0)]
    result = [-1]

    def dfs(r, c, length):
        if (r, c) == dest:
            result[0] = max(result[0], length)
            return

        for dr, dc in directions:
            nr, nc = r+dr, c+dc
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] != 0 ar
                visited[nr][nc] = True
                dfs(nr, nc, length+1)
                visited[nr][nc] = False  # backtrack
```

```
    if grid[0][0] != 0:
        visited[0][0] = True
        dfs(0, 0, 0)

    return result[0]

grid = [[1,1,1],[1,0,1],[1,9,1]]
print(longest_path(grid))  # Should find the longest path to 9
```

---

## Pattern 2: Array / List Problems

### Two Sum

```
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

print(two_sum([2,7,11,15], 9))  # [0, 1]
```

### Find Maximum Subarray (Kadane's Algorithm)

```
def max_subarray(nums):
    max_sum = current_sum = nums[0]
    for num in nums[1:]:
        current_sum = max(num, current_sum + num)
        max_sum = max(max_sum, current_sum)
    return max_sum

print(max_subarray([-2,1,-3,4,-1,2,1,-5,4]))  # 6
```

### Find Duplicates in Array

```
def find_duplicates(nums):
    seen = set()
```

```
    duplicates = []
    for n in nums:
        if n in seen:
            duplicates.append(n)
        seen.add(n)
    return duplicates
```

---

## Pattern 3: String Problems

#### Check if String is Palindrome

```
def is_palindrome(s):
    s = s.lower().replace(' ', '')
    return s == s[::-1]
```

#### Count Word Frequencies (Very relevant for NLP)

```
from collections import Counter

def word_frequency(text):
    words = text.lower().split()
    return Counter(words)

text = "the cat sat on the mat the cat"
print(word_frequency(text))
# Counter({'the': 3, 'cat': 2, ...})
```

#### Anagram Check

```
def is_anagram(s, t):
    return Counter(s) == Counter(t)
```

---

## Pattern 4: Sorting & Searching

#### Binary Search

```
def binary_search(arr, target):
    left, right = 0, len(arr)-1
```

```
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

---

## Pattern 5: Stack & Queue Problems

#### Valid Parentheses

```
def is_valid(s):
    stack = []
    mapping = {')': '(', '}': '{', ']': '['}
    for char in s:
        if char in mapping:
            top = stack.pop() if stack else '#'
            if mapping[char] != top:
                return False
        else:
            stack.append(char)
    return not stack

print(is_valid("()[]{}"))  # True
print(is_valid("([)]"))    # False
```

---

# 📌 Section 2: Python & scikit-learn ML Code Questions

> Expect code completion or "what does this code do?" questions.

---

## Q1. Write a complete ML pipeline in Python.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.pipeline import Pipeline

# Load data
df = pd.read_csv('data.csv')
X = df.drop('target', axis=1)
y = df['target']

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', RandomForestClassifier(n_estimators=100, random_state=42))
])

# Train
pipeline.fit(X_train, y_train)

# Evaluate
y_pred = pipeline.predict(X_test)
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))

# Cross-validation
cv_scores = cross_val_score(pipeline, X, y, cv=5, scoring='f1_weighted')
print(f"CV F1: {cv_scores.mean():.3f} ± {cv_scores.std():.3f}")
```

---

## Q2. How do you handle missing values in pandas?

```
import pandas as pd
import numpy as np

df = pd.read_csv('data.csv')

# Check missing values
print(df.isnull().sum())
```

```python
print(df.isnull().mean() * 100)  # percentage

# Drop rows/columns with too many nulls
df.dropna(thresh=len(df)*0.5, axis=1, inplace=True)  # drop cols >50% nul

# Fill numeric with median (robust to outliers)
num_cols = df.select_dtypes(include=np.number).columns
df[num_cols] = df[num_cols].fillna(df[num_cols].median())

# Fill categorical with mode
cat_cols = df.select_dtypes(include='object').columns
df[cat_cols] = df[cat_cols].fillna(df[cat_cols].mode().iloc[0])

# Advanced: sklearn Imputer
from sklearn.impute import SimpleImputer, KNNImputer
imputer = KNNImputer(n_neighbors=5)
df_imputed = pd.DataFrame(imputer.fit_transform(df[num_cols]), columns=nu
```

---

## Q3. How do you perform feature scaling?

```python
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustSca

# StandardScaler: (x - mean) / std  → mean=0, std=1
# Best for: Gaussian-like data, algorithms that use distance (SVM, KNN, P
scaler = StandardScaler()

# MinMaxScaler: (x - min) / (max - min) → [0,1]
# Best for: Neural networks, image pixel values
scaler = MinMaxScaler()

# RobustScaler: uses median and IQR → robust to outliers
scaler = RobustScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)  # NEVER fit on test set!
```

---

## Q4. How do you implement TF-IDF from scratch?

```python
import numpy as np
from collections import Counter
```

```
import math

def compute_tfidf(corpus):
    # Tokenize
    tokenized = [doc.lower().split() for doc in corpus]

    # Vocabulary
    vocab = list(set(word for doc in tokenized for word in doc))

    N = len(corpus)

    # Document frequency
    df = {}
    for word in vocab:
        df[word] = sum(1 for doc in tokenized if word in doc)

    # IDF
    idf = {word: math.log(N / df[word]) for word in vocab}

    # TF-IDF matrix
    tfidf_matrix = []
    for doc in tokenized:
        tf = Counter(doc)
        doc_tfidf = {word: (tf[word]/len(doc)) * idf[word] for word in vc
        tfidf_matrix.append(doc_tfidf)

    return tfidf_matrix

corpus = ["I love machine learning", "machine learning is great", "I love
result = compute_tfidf(corpus)
```

---

## Q5. How do you implement K-Means from scratch?

```
import numpy as np

class KMeans:
    def __init__(self, k=3, max_iters=100):
        self.k = k
        self.max_iters = max_iters

    def fit(self, X):
        # Initialize centroids randomly
```

```
        idx = np.random.choice(len(X), self.k, replace=False)
        self.centroids = X[idx]

        for _ in range(self.max_iters):
            # Assign clusters
            labels = self._assign(X)

            # Update centroids
            new_centroids = np.array([X[labels==i].mean(axis=0) for i in

            # Check convergence
            if np.allclose(self.centroids, new_centroids):
                break
            self.centroids = new_centroids

        return labels

    def _assign(self, X):
        distances = np.array([[np.linalg.norm(x - c) for c in self.centro
        return np.argmin(distances, axis=1)
```

---

## Q6. How do you detect and handle outliers?

```
import numpy as np
import pandas as pd

# Method 1: IQR (Interquartile Range)
def remove_outliers_iqr(df, col):
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    return df[(df[col] >= lower) & (df[col] <= upper)]

# Method 2: Z-score
from scipy import stats
z_scores = np.abs(stats.zscore(df['column']))
df_clean = df[z_scores < 3]  # remove > 3 standard deviations

# Method 3: Isolation Forest (ML-based)
from sklearn.ensemble import IsolationForest
```

```
clf = IsolationForest(contamination=0.05)  # expect 5% outliers
outlier_pred = clf.fit_predict(X)
X_clean = X[outlier_pred == 1]  # 1=inlier, -1=outlier
```

---

# 📌 Section 3: Behavioral / HR Questions

---

## Q1. Tell me about yourself.

**Template Answer:**

"I'm Anik Paul, an Independent Researcher with 2 years of hands-on experience in machine learning, deep learning, computer vision, and NLP. I completed my B.Sc. in Computer Science from Premier University, Chattogram in 2025. My academic and research work has been deeply practical — I've built sentiment analysis systems for Bangla text, working with sparse features like BoW and TF-IDF, embedding models like Word2Vec and FastText, and ensemble classifiers including Random Forest, XGBoost, and LightGBM. I'm currently a first author on a manuscript submitted to an MDPI Q1 journal. Beyond NLP, I'm working on real-time vision systems using transformers and YOLO, and a privacy-preserving computer vision project for blind assistance. I'm excited about this role at ReliSource because it directly aligns with the MLOps and production deployment side of ML, which is the next frontier I want to grow into."

---

## Q2. Describe a challenging ML project and how you handled it.

**Template Answer:**

"In my Bangla sentiment analysis project, the biggest challenge was the agglutinative morphology of Bangla — a single root word can take thousands of surface forms, making traditional BoW representations very sparse and unreliable. I handled this by combining multiple feature types: I horizontally stacked BoW, TF-IDF, and character n-gram features to create richer representations. I also used FastText, which naturally handles subword information and out-of-vocabulary words. For feature selection, I applied Chi-squared and ANOVA F tests to identify the most discriminative features. I then ran multiple classifiers — Random Forest, XGBoost, LightGBM, SVM, and Logistic Regression — and used ensemble strategies like voting, stacking, and hybrid combinations to squeeze out the best performance. The key lesson was that no single approach solves linguistic complexity; combining sparse and dense features gave the best results."

---

## Q3. Why do you want to work at ReliSource?

**Template Answer:**

"ReliSource works with Fortune 500 clients across healthcare, logistics, and fintech — domains where ML model reliability and MLOps maturity really matter. The role involves building production ML pipelines and monitoring models in deployment, which is exactly the area I want to deepen. My research background has been strong on the modeling side, and ReliSource gives me the opportunity to develop the engineering discipline — CI/CD for ML, Azure deployment, scalable pipelines — alongside experienced engineers. The global client exposure is also very attractive since I want to work in English-speaking professional environments."

---

## Q4. How do you stay updated with ML advancements?

**Answer:**

"I regularly follow arXiv for preprints, especially the cs.LG, cs.CV, and cs.CL sections. I read Papers With Code to see SOTA benchmarks and implementations. I follow key researchers on Twitter/X and read blog posts from Hugging Face, Google AI, and Andrej Karpathy. For applied knowledge, I watch fast.ai lectures and read the MLOps Community blog. In my own research, keeping up with new architectures like vision transformers and efficient attention mechanisms is part of my day-to-day work."

---

## Q5. Where do you see yourself in 3–5 years?

**Answer:**

"In 3 years, I see myself as a confident ML engineer who can design and deploy production-grade ML systems end-to-end — from data pipeline to model monitoring. I want to grow into a technical lead role where I can mentor junior engineers and drive architecture decisions. In 5 years, I'd like to be working on cutting-edge applied AI problems, potentially in low-resource language AI or multimodal systems, contributing to both research and real-world deployments."

---

## Q6. How do you handle a situation where your model performs well in training but poorly in production?

**Answer:**

"This is a data drift or leakage problem. I'd start by analyzing the feature distributions — comparing training data statistics with what the model sees in production using tools like Evidently AI or custom monitoring. If drift is detected, I check whether features have changed semantically or statistically. I'd also review the training pipeline for leakage — sometimes preprocessing steps like scaling were fit on all data rather than just training data. If the model is genuinely decaying, I'd trigger a retraining cycle,

potentially with a mix of historical and recent data. For ongoing prevention, I'd set up automated alerts when key metrics drop below a threshold."

---

### Q7. Describe a time you had to communicate a complex technical concept to a non-technical stakeholder.

**Answer:**

"During my research assistant work supporting postgraduate students, I often had to explain why a model with 95% accuracy on an imbalanced dataset was actually performing poorly on the minority class. I used a simple analogy: imagine a disease that affects 5% of people. A model that always says 'healthy' would be 95% accurate, but useless. I visualized the confusion matrix and precision-recall curves, and explained that for their research goal — detecting the minority case — recall on that class was the metric that mattered. They shifted their evaluation approach based on that conversation. I've found that visual aids and concrete analogies work much better than mathematical explanations with non-technical people."

---

## 📌 Section 4: Quick Reference — Common Interview Gotchas

---

### When to use which algorithm?

| Scenario | Go-To Algorithm | Why |
|---|---|---|
| High-dimensional text data | Logistic Regression, SVM (linear) | Fast, works well with sparse features |
| Tabular data, competitions | XGBoost / LightGBM | Best performance out-of-the-box |
| Need interpretability | Decision Tree, Linear/Logistic Regression | Directly explainable |
| Noisy data, many features | Random Forest | Robust, handles irrelevant features |
| Image classification | CNN / ResNet / EfficientNet | Spatial hierarchy of features |
| Sequential text | LSTM / Transformer / BERT | Captures sequence context |
| Clustering | K-Means (spherical), DBSCAN (arbitrary shape) | Different cluster assumptions |

| Scenario | Go-To Algorithm | Why |
|---|---|---|
| Anomaly detection | Isolation Forest, One-Class SVM | Works without labeled anomalies |

---

## Key Python Libraries Cheat Sheet

```
# Data manipulation
import pandas as pd
import numpy as np

# ML
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClas
from sklearn.linear_model import LogisticRegression, Ridge, Lasso
from sklearn.svm import SVC, SVR
import xgboost as xgb
import lightgbm as lgb

# Deep learning
import torch
import torch.nn as nn
import tensorflow as tf
from transformers import AutoModelForSequenceClassification, AutoTokenize

# NLP
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectori
from gensim.models import Word2Vec, FastText
import nltk

# Computer vision
import cv2
from torchvision import models, transforms
from ultralytics import YOLO

# Metrics
from sklearn.metrics import (
    accuracy_score, f1_score, precision_score, recall_score,
    roc_auc_score, confusion_matrix, classification_report,
    mean_squared_error, mean_absolute_error, r2_score
)

# MLOps
import mlflow
```

```
mlflow.start_run()
mlflow.log_metric("f1", f1_score(y_test, y_pred))
mlflow.sklearn.log_model(model, "model")
```

---

### Common Mistake Traps in Interviews

| Trap | What they ask | Correct Answer |
|---|---|---|
| Accuracy on imbalanced data | "95% accuracy — is the model good?" | Not necessarily. Check F1/AUC on minority class |
| Scaling before or after split | "When do you scale?" | AFTER splitting — fit scaler on train only |
| Correlation vs causation | "Feature X correlates with Y, can you drop X?" | No, correlation ≠ causation; X may still be useful |
| k-fold on time series | "Use k-fold CV?" | No — use TimeSeriesSplit to prevent future leakage |
| Hyperparameters vs parameters | "What's a hyperparameter?" | Not learned from data (learning rate, n_estimators) |

---

## 📌 Section 5: Quick 60-Second Answers (For Rapid-Fire Interview)

- **What is regularization?** → Penalty on model complexity to prevent overfitting. L1 gives sparse models, L2 shrinks all weights.
- **What is the kernel trick?** → Maps data to higher dimensions implicitly, without computing the transformation explicitly. Allows SVM to find non-linear boundaries.
- **What is SMOTE?** → Synthetic Minority Oversampling Technique. Creates synthetic samples for minority class by interpolating between existing samples.
- **What is early stopping?** → Stop training when validation loss stops decreasing. Prevents overfitting in neural networks.
- **What is attention?** → Mechanism that lets a model focus on relevant parts of the input when making predictions. Core of transformers.
- **Difference between precision and recall?** → Precision = correct out of all predicted positives. Recall = correct out of all actual positives.
- **What is ROC-AUC?** → Probability that the model ranks a random positive higher than a random negative. AUC=1 is perfect, 0.5 is random.
- **What is the difference between GBM and XGBoost?** → XGBoost adds regularization, uses second-order gradients, column subsampling, and is much faster.

- **What is word embedding?** → Dense vector representation of words where semantically similar words have similar vectors.
- **What is fine-tuning?** → Taking a pre-trained model and further training it on domain-specific data with a low learning rate.

---